Personal Expense CalCulator Documentation

Course: Introduction to Python and Machine Learning Foundations

Author: Syed Shahwaiz Abbas Rizvi

What problem is it solving?

Manual expense tracking on paper or spreadsheets is slow, error prone, and causes frequent budget overruns due to untracked spending.

How is it solving ?

Our Python tool automates expense tracking, validates entries, categorizes spending, and sends alerts when budget limits are reached.

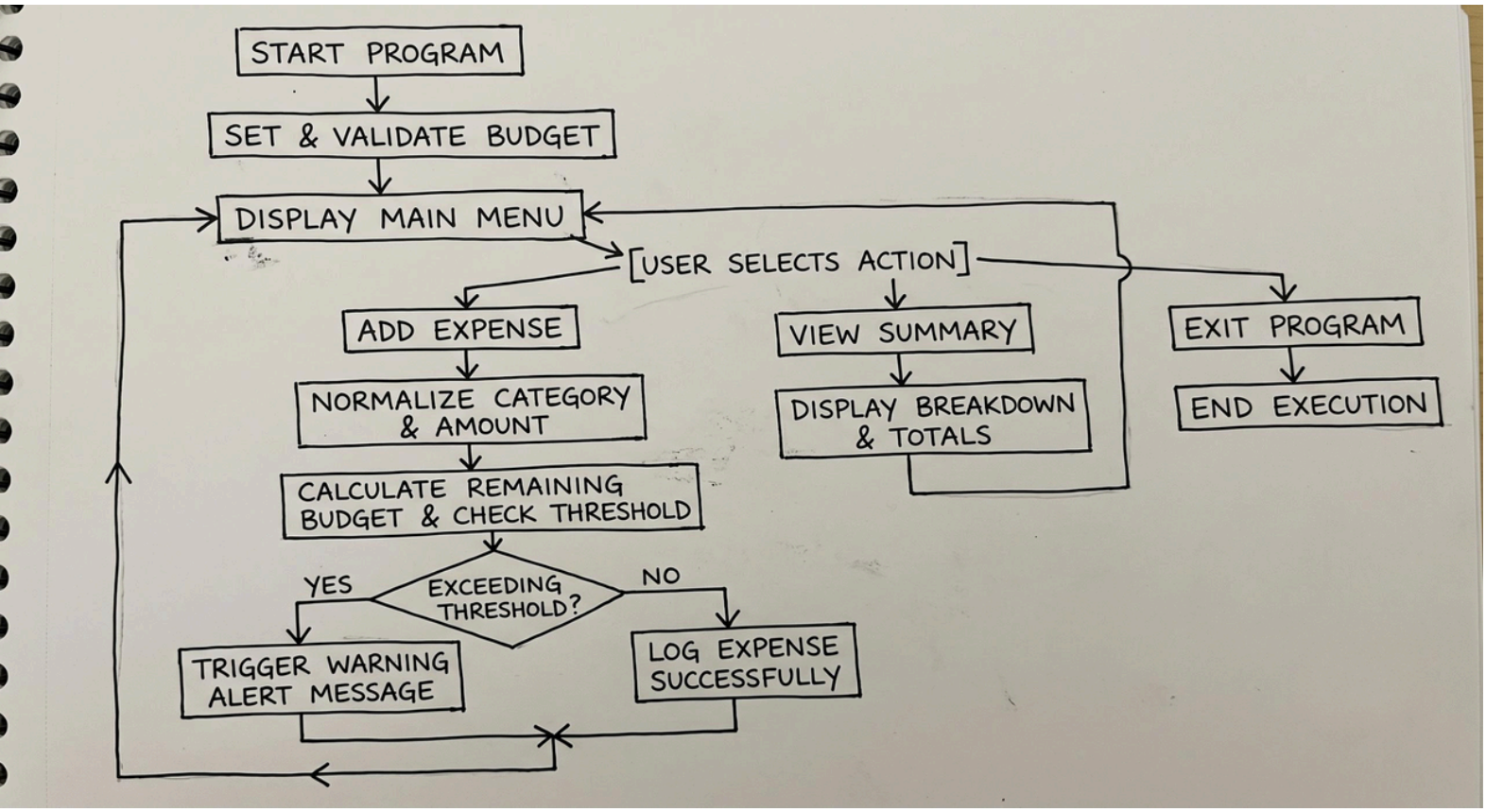
Algorithm (step-by-step solution written in English) :

- Initialize System: Set up storage structures (dictionaries and lists).
- Set Budget: Prompt the user to enter their total budget
- Display Main Menu: Present clear navigation options (e.g., Add Expense, View Summary, Check Remaining Budget, Exit).

Input Processing & Normalization:

- Normalize the input string (e.g., converting text to uppercase or standard case) so entries like "food" and "Food" are mapped into a single unified category.
- Budget Calculation & Alert Triggering:
- Deduct the expense amount from the remaining budget pool..
- If total spending exceeds defined threshold percentages (or the total limit), display an immediate warning/alert message to the user.
- Summary Generation: Calculate category totals and produce a breakdown report upon user request.
- Exit Mechanism: Safely terminate the program loop when selected by the user.

Visualization (flowchart) :



The issues that I spent most of my time on :

The most time taking was to spent on setting up and implementing input validations and constructing reliable predefined function routines. Preventing application crashes due to invalid user inputs

what strategies I used to fix the issue? :

- Applied string normalization methods (.upper() / .strip())
- Implemented **try&except** blocks along with while loops to catch ValueError

If you had to do it again, what mistakes would you avoid? :

I would avoid use on deep nested loops. Instead, I would heavily use built-in functions and libraries to minimize code complexity, reduce lines of code.

What things I am the most proud of?

I am most proud of the interactive Main Menu interface and clean reporting formatting.

Do you think users will find these useful?

Yes, users will find this application highly valuable. Time is a valuable factor; manual bookkeeping is inefficient.

There is quote:

TIME IS FREE, BUT IT'S PRICELESS.ONCE YOU'VE LOST IT YOU CAN NEVER GET IT BACK.

What features have I missed out here?

- Automatic Date/Time tracking per transaction record.
- Account mapping (e.g., credit card, cash, or specific bank accounts).

What steps would I take to improve this?

- Digital Banking Integration: Integrating it to Automatically add transactions done by Banking app
- Notification System: Real-time SMS or push notifications when approaching budget Deadline
- Reporting Summaries: Sending daily or weekly financial summaries to the user

Do you think this is a good fit for a website?

YES! Especially for **business websites** to help them calculate profit and loss margins, manage investments.

What are your key learnings from this entire experience?

The key element from this project was discovering how simple Python concepts like data structures, functions, and string methods can be combined to build tools that solve daily operational problems and save meaningful time.